Product Review Sentiment Analysis

A Project Report

In the partial fulfillment of the award of the degree of

B. Tech

at

Ardent Computech Pvt. Ltd

Submitted by

Anirban Sarkar, Aniket Das, Anushka Paul
Arnab Bhattacharya, Ankan Dey

Narula Institute Of Technology

Certificate from the Mentor

This is to certify that **Anirban Sarkar, Aniket Das, Anushka Paul, Arnab Bhattacharya, Ankan Dey** has completed the project titled **Product Review Sentiment Analysis** under my supervision during the period from **16/02/26** to **26/02/26** which is in partial fulfillment of requirements for the award of the **B.Tech** and submitted to Department **CSE** of **Narula Institute Of Technology**

_____

Signature of the Mentor

Date:

Acknowledgment

I take this opportunity to express my deep gratitude and sincerest thanks to my project mentor, **TRIDIB KUNDU** for giving the most valuable suggestions, helpful guidance, and encouragement in the execution of this project work.

I would like to give a special mention to my colleagues. Last but not least I am grateful to all the faculty members of the **Ardent Computech Pvt. Ltd.** for their support.

Abstract

This project presents a deep learning–based three-class sentiment classification system for Amazon product reviews, categorizing text into **Negative, Neutral, and Positive** sentiments. A key challenge addressed in this work is the severe class imbalance inherent in real-world review datasets, where positive reviews overwhelmingly dominate minority classes.

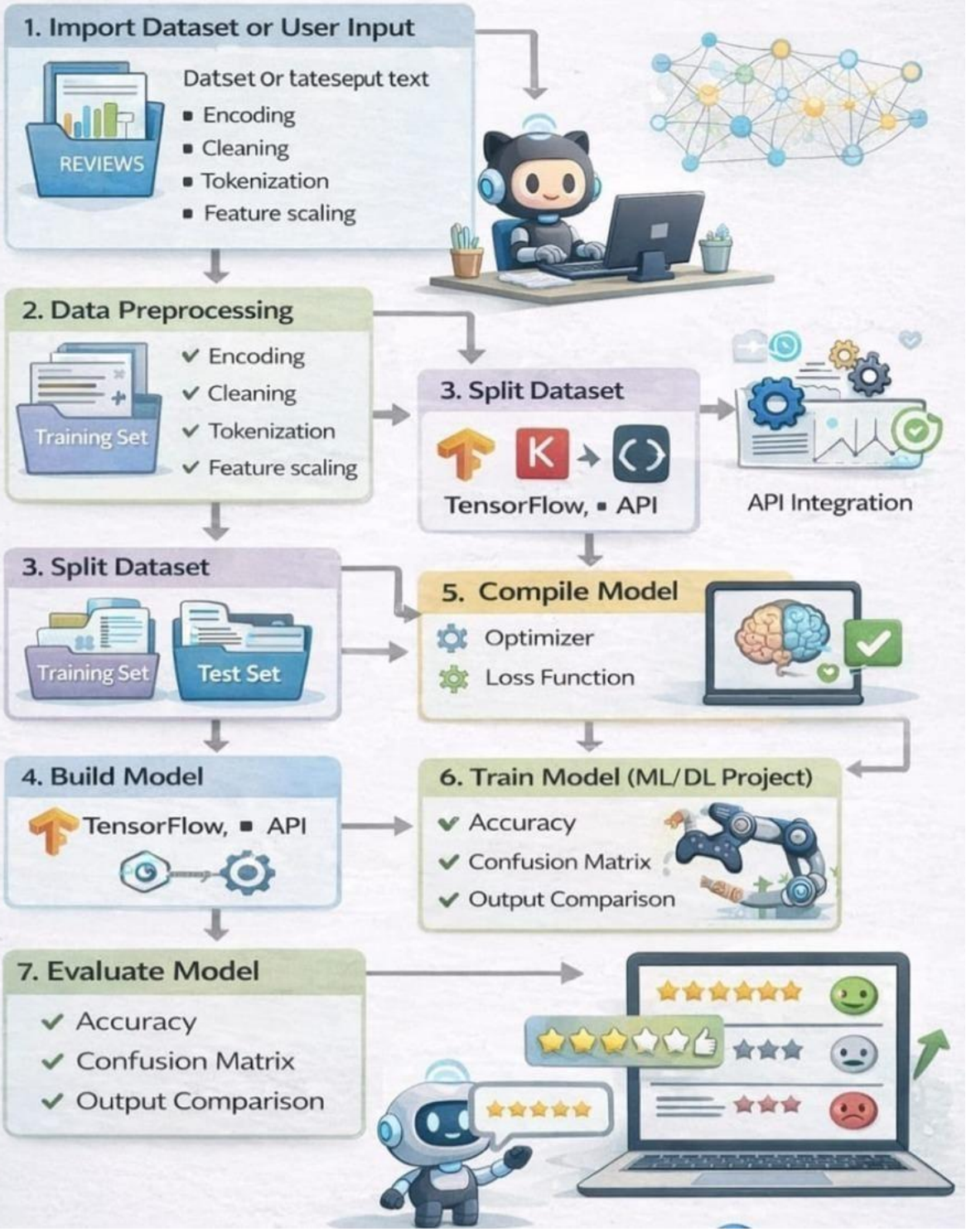
To tackle this issue, a structured four-layer imbalance mitigation strategy is implemented across the entire pipeline. First, stratified data splitting ensures proportional class representation in training, validation, and test sets. Second, partial random oversampling increases minority-class samples to 70% of the majority class to improve balance while avoiding overfitting. Third, Focal Loss with inverse-frequency alpha weighting is employed during training to prioritize hard and minority examples. Finally, per-class decision threshold calibration is applied at inference time to maximize macro F1-score and enhance minority recall.

The model architecture consists of an Embedding layer followed by stacked Bidirectional LSTM layers and a Self-Attention mechanism, enabling contextual understanding and selective focus on sentiment-indicative words such as negations and intensifiers. Regularization techniques including Spatial Dropout, Batch Normalization, Early Stopping, and learning rate scheduling further improve generalization.

Performance is evaluated using imbalance-aware metrics including Balanced Accuracy, Macro F1-score, Matthews Correlation Coefficient (MCC), ROC curves, and Precision-Recall curves. Experimental results demonstrate that the proposed multi-layer defence strategy significantly improves minority-class detection while maintaining strong overall performance. This work highlights the importance of combining data-level, algorithm-level, and inference-level techniques to build robust sentiment classifiers for real-world imbalanced datasets.

Product Review Sentiment Analysis

Build a deep learning model to classify product reviews based on sentiment.



Chapters	Page no.

- Abstract 4

Chapter 1:

- Introduction 8
- Background 9
- Importance of Deep Learning in this project 10
- Problem Statement 10
- Objectives 11

Chapter 2:

- Related Work 12

Chapter 3:

- Literature Survey 13-15
- Some Deep learning approaches in product review sentiment analysis 16
- Introduction to deep learning and how it works 17-19

Chapter 4:

- Methodology 20-23
  6.0 Data Acquisition
  6.1 Exploratory Data Analysis(EDA)
  6.2 Text Preprocessing
  6.3 Class imbalance handling
  6.4 Tokenization and sequencing
  6.5 Model Architechture

6.6 Training Strategy
6.7 Evaluation Metrices

- Implementation and experimental results 24-34

Chapter 5:

- Results & Discussion 35-36

Chapter 6:

- Frontend Implementation 37-40
- Backend Implementation

Chapter 7:

- Conclusion & Future Work 41-42

Chapter 8:

- References 42

Chapter 1

Introduction

With the rapid growth of e-commerce platforms such as Amazon, millions of product reviews are generated daily, providing valuable insights into customer satisfaction, product quality, and purchasing behaviour. These reviews strongly influence consumer decisions and business strategies. However, manually analyzing such massive volumes of textual data is impractical, creating the need for automated sentiment analysis systems powered by Natural Language Processing (NLP) and Deep Learning.

Sentiment analysis aims to classify textual content based on emotional tone, typically into categories such as Positive, Neutral, or Negative. While many traditional machine learning approaches have been applied to this problem, real-world review datasets present a significant challenge: severe class imbalance. In most e-commerce datasets, positive reviews dominate, while neutral and especially negative reviews are comparatively rare. This imbalance often causes standard models to achieve high overall accuracy while performing poorly on minority classes — particularly negative sentiment, which is often the most critical for business insights.

To address this challenge, this project proposes a deep learning–based three-class sentiment classification system specifically designed to handle class imbalance effectively. The system integrates a structured multi-layer defense strategy across data preparation, model training, and inference stages. Techniques such as stratified data splitting, controlled oversampling, Focal Loss with inverse-frequency weighting, and per-class decision threshold calibration are applied to ensure balanced learning and improved minority-class detection. The model architecture employs stacked Bidirectional Long Short-Term Memory (BiLSTM) networks combined with a Self-Attention mechanism to capture contextual dependencies and highlight sentiment-bearing words such as negations and intensifiers. The system is evaluated using imbalance-aware performance metrics including Macro F1-score, Balanced Accuracy, and Matthews Correlation Coefficient (MCC), ensuring fair assessment across all sentiment classes.

By combining data-level, algorithm-level, and inference-level strategies, this project demonstrates how robust sentiment classification can be achieved in highly imbalanced real-world review datasets.

Background

The exponential growth of online shopping platforms such as Amazon has led to the generation of massive volumes of user-generated content in the form of product reviews. These reviews play a crucial role in influencing purchasing decisions, shaping brand reputation, and providing feedback for product improvement. As a result, extracting meaningful insights from textual reviews has become an important research area in **Natural Language Processing (NLP)** and **Sentiment Analysis**.

Traditionally, sentiment analysis was performed using classical machine learning techniques such as Naïve Bayes, Support Vector Machines (SVM), and Logistic Regression with handcrafted features like TF-IDF or Bag-of-Words. While these approaches achieved reasonable results, they struggled to capture contextual dependencies, sarcasm, negations, and subtle sentiment cues present in real-world text data.

With advancements in deep learning, models such as Recurrent Neural Networks (RNNs), Long Short-Term Memory (LSTM) networks, and Bidirectional LSTMs (BiLSTMs) became popular for text classification tasks. These models can learn sequential dependencies and contextual meaning directly from raw text, eliminating the need for extensive manual feature engineering. Attention mechanisms further enhanced performance by allowing models to focus selectively on important words within a sentence.
Despite these advancements, a major unresolved challenge in real-world sentiment datasets is **class imbalance**. In most e-commerce review datasets, positive reviews significantly outnumber neutral and negative ones. This imbalance can bias models toward predicting the majority class, resulting in poor detection of minority classes—especially negative reviews, which are often the most critical for identifying product issues and customer dissatisfaction.
Research in imbalanced learning has introduced multiple strategies to address this issue, including:

- **Data-level techniques** (oversampling, under sampling, synthetic data generation)
- **Algorithm-level techniques** (class-weighted loss functions, Focal Loss)
- **Evaluation-level techniques** (macro F1-score, balanced accuracy instead of plain accuracy)
- **Threshold tuning methods** to improve minority recall

This project is grounded in these research developments and integrates multiple imbalance-handling strategies into a unified pipeline. By combining modern deep learning architectures (BiLSTM + Self-Attention) with structured imbalance mitigation techniques, the project aims to build a sentiment classifier that performs robustly across all classes, rather than achieving inflated accuracy driven only by the dominant positive class.
In summary, the background of this project lies at the intersection of deep learning–based NLP and imbalanced classification research, addressing a practical and widely encountered challenge in real-world e-commerce review analysis.

Importance of Deep Learning in this project

Deep Learning is the core technology behind this sentiment classification system. Unlike traditional machine learning models that rely on handcrafted features, deep learning automatically learns complex language patterns from raw text. This allows the model to understand context, capture important sentiment cues, and handle large-scale imbalanced datasets efficiently, making it ideal for classifying real-world Amazon product reviews.

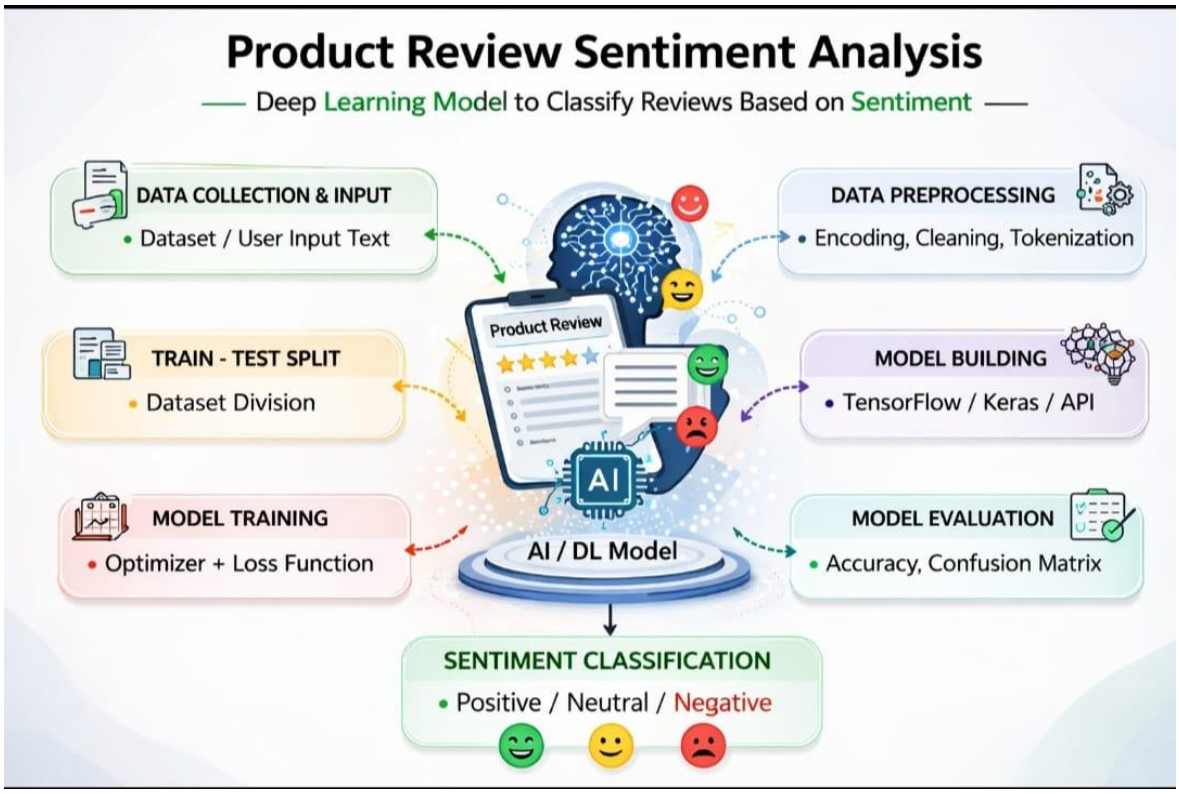
- **Context & Sequential Understanding:** BiLSTM captures both past and future context, correctly interpreting phrases like *"The product is not good"* vs *"The product is good"*.
- **Automatic Feature Extraction:** Word embeddings convert text into dense vectors, capturing semantic similarity and eliminating manual feature engineering.
- **Attention to Key Words:** Self-Attention highlights sentiment-bearing tokens such as negations (*not, never*), intensifiers (*very, extremely*), and strong words (*awful, amazing*).
- **Effective Class Imbalance Handling:** Focal Loss emphasizes hard-to-classify and minority-class examples, working with oversampling and threshold calibration.
- **Scalability & High Performance:** Deep networks handle large vocabularies and datasets, enabling real-time deployment for large-scale review monitoring.
- **Better Generalization:** Dropout, batch normalization, and early stopping prevent overfitting, ensuring strong performance on unseen data.

Problem Statement

Sentiment analysis of product reviews is essential for understanding customer satisfaction, product quality, and user experience on platforms like Amazon. However, real-world review datasets are **highly imbalanced**, with positive reviews far outnumbering neutral and negative ones. This makes it challenging for standard models to correctly classify minority classes, particularly negative reviews, which are critical for identifying product issues.
The problem is to develop a **deep learning–based sentiment classification system** capable of handling this imbalance. The system must process raw review text through cleaning, tokenization, and stop-word removal, use **stratified train/validation/test splits**, and leverage a **BiLSTM + Self-Attention architecture** to capture context and sequential information. Training incorporates **Focal Loss, class weights, and early stopping** to improve learning for minority classes, while evaluation uses both traditional and imbalance-aware metrics (accuracy, balanced accuracy, macro F1, MCC). Finally, **per-class threshold calibration** ensures robust predictions across all sentiment categories.
In essence, the challenge is to build a **scalable, reliable, and imbalance-aware model** that accurately classifies Negative, Neutral, and Positive reviews, providing actionable insights from real-world datasets.

Objectives

1. **Develop a sentiment classification model** to categorize Amazon product reviews into **Negative, Neutral, and Positive** classes.
2. **Handle severe class imbalance** using strategies such as stratified splits, partial oversampling, class-weighted Focal Loss, and per-class threshold calibration.
3. **Preprocess textual data effectively**, including cleaning, tokenization, stop-word removal, and handling negations to retain key sentiment signals.
4. **Build a deep learning architecture** combining BiLSTM and Self-Attention to capture contextual dependencies and highlight important sentiment-bearing words.
5. **Train the model efficiently** with early stopping, class weights, and learning rate adjustments to prevent overfitting and improve minority-class performance.
6. **Evaluate the model using robust metrics**, including accuracy, balanced accuracy, macro F1-score, Matthews Correlation Coefficient (MCC), ROC, and Precision-Recall curves.
7. **Deploy a reliable inference pipeline** with per-class decision threshold calibration to maximize recall for minority classes while maintaining overall accuracy.



Chapter 2

Related Work

Several studies have explored sentiment analysis of product reviews using both traditional machine learning and deep learning approaches. An earlier conference study on Amazon product reviews proposed a sentence-level and review-level sentiment classification framework using large-scale datasets. The authors performed preprocessing steps such as POS tagging, negation handling, and sentiment token extraction, followed by feature vector construction based on sentiment scores. Classical classifiers including Naïve Bayes, Support Vector Machines (SVM), and Random Forest were used for polarity classification. While the study achieved promising F1-scores, the approach relied heavily on handcrafted features and lexicon-based scoring mechanisms, limiting its ability to capture deeper contextual relationships.

Another research work focused on evaluating Amazon reviews by categorizing them into positive, neutral, and negative classes using the TextBlob library in Python. The study emphasized exploratory data analysis and word frequency statistics, revealing that product quality and price were primary consumer concerns. Although effective for basic polarity detection, the lexicon-based sentiment scoring approach struggled with contextual complexity, sarcasm, and implicit sentiment expressions.

More recently, a study published in Scientific Reports proposed a hybrid deep learning architecture combining Bidirectional Gated Recurrent Units (BiGRU) for feature extraction and an RNN-based Long Short-Term Memory (LSTM) classifier for multi-class sentiment prediction. The model incorporated extensive preprocessing techniques and was evaluated against CNN, MLP, CapsuleNet, and GAN-based models, achieving superior classification performance. However, the study primarily focused on architectural improvements and comparative evaluation, without explicitly addressing class imbalance issues in depth.

In contrast to these approaches, the present work leverages a BiLSTM with Self-Attention mechanism and integrates imbalance-aware techniques such as Focal Loss, class weighting, and decision threshold calibration to enhance minority-class recall and overall balanced performance.

Chapter 3

Literature Survey

Sentiment analysis of product reviews has been extensively explored using both traditional machine learning and deep learning techniques. Early research on Amazon product review datasets primarily relied on classical machine learning classifiers such as Naïve Bayes, Support Vector Machines (SVM), Decision Trees, and Random Forest. These models depended heavily on handcrafted features, TF-IDF representations, bag-of-words models, and sentiment lexicons for polarity classification. Although these approaches achieved moderate accuracy, they struggled to capture contextual dependencies, sarcasm, and long-range semantic relationships within complex review texts.

Subsequently, lexicon-based natural language processing (NLP) tools such as TextBlob and VADER were introduced for polarity detection and rule-based scoring. These methods were computationally efficient and easy to implement, making them suitable for basic sentiment categorization and consumer behaviour analysis. However, due to their rule-driven nature, they lacked deep contextual understanding and were limited when handling ambiguous or domain-specific expressions in product reviews.

With the advancement of deep learning, researchers began employing neural network architectures such as Convolutional Neural Networks (CNN), Recurrent Neural Networks (RNN), and Long Short-Term Memory (LSTM) networks for sentiment classification. A study published in The Journal of Supercomputing (2022) performed a comparative analysis of CNN, RNN, and BiLSTM models using various word embeddings such as Word2Vec, FastText, BERT, and RoBERTa. The study demonstrated that deep neural models significantly outperformed traditional machine learning techniques, with the CNN-RNN-BiLSTM hybrid model achieving the highest accuracy and F1-score. The research also highlighted the effectiveness of data augmentation techniques in improving classification robustness.

More recently, a 2025 study published in Scientific Reports proposed a hybrid deep learning framework combining Bidirectional GRU (BiGRU) and LSTM networks for multi-class sentiment classification. The hybrid architecture enhanced contextual feature extraction by capturing both forward and backward dependencies in review sequences. Experimental results confirmed superior performance compared to standalone deep learning baselines, emphasizing the importance of hybrid architectures for improved sentiment prediction accuracy.

Despite these advancements, most existing studies primarily focused on improving model architectures and embedding strategies. Limited attention has been given to addressing severe class imbalance in multi-class sentiment datasets, where positive reviews significantly outnumber neutral and negative ones. Handling imbalance effectively remains a critical challenge for achieving balanced performance across all sentiment categories, motivating the need for advanced imbalance-aware training strategies in sentiment analysis systems.

SL NO	AUTHOR	JOURNAL	TITLE OF THE PAPER	METHOD AND YEAR	OUTCOME
1.	Pawanjit Singh Ghatora, Seyed Ebrahim Hosseini, Shahbaz Pervez, Muhammad Javed Iqbal, and Nabil Shaukat	Big Data and Cognitive Computing (MDPI)	Sentiment Analysis of Amazon Product Reviews Using Deep Learning Models	Hybrid BiGRU–LSTM sentiment model. 2024	The proposed BiGRU–LSTM model achieved higher accuracy and superior contextual sentiment classification performance compared to traditional and baseline deep learning models.
2.	Xing Fang , Justin Zhan	Journal of Big Data	Sentiment Analysis Using Product Review Data	Lexicon-based sentiment token extraction with sentiment score computation and classification using Naïve Bayes, Random Forest, and SVM. 2015	The proposed sentiment polarity categorization framework achieved strong F1-scores (above 0.8 for sentence-level classification), with Random Forest performing best among the classifiers.

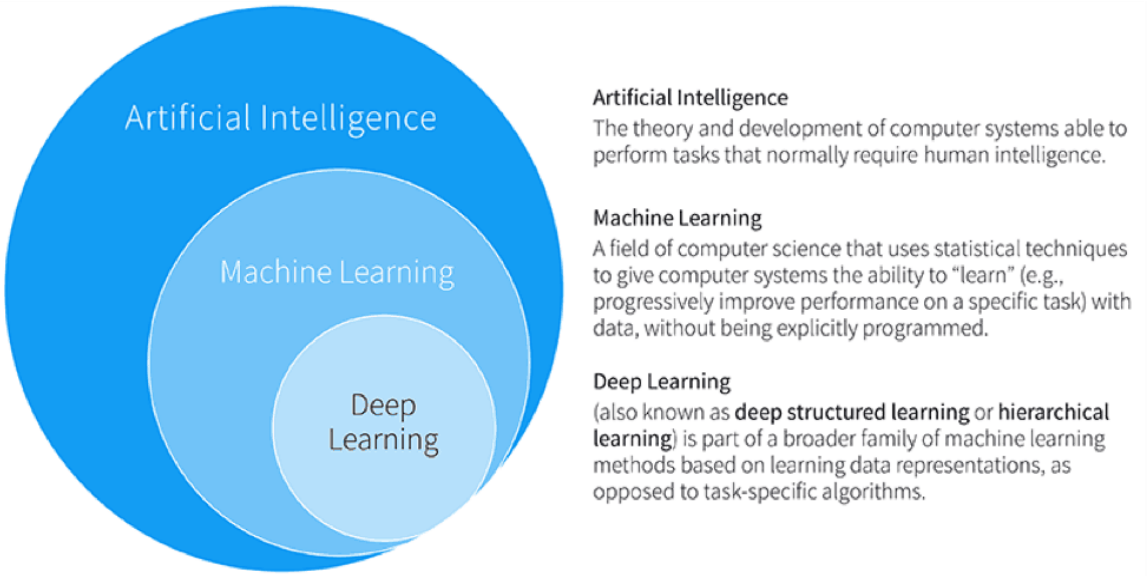
3.	L. Godlin Atlas , Daniel Arockiam Arvindhan Muthusamy, Balamurugan, Balusamy Shitharth, Selvarajan Taher Al-Shehari	Scientific Reports	modernized approach to sentiment analysis of product reviews using BiGRU and RNN based LSTM deep learning models	Hybrid BiGRU–LSTM deep learning sentiment classification model. 2021	The BiGRU–LSTM hybrid architecture achieved superior sentiment classification accuracy by effectively capturing both contextual and sequential dependencies in product reviews.
4.	Mohammad Irsad, Ashish Khare	International Journal on Recent and Innovation Trends in Computing and Communication (IJRITCC)	A Deep Learning based Model using Review Associated Feature Extraction Approach for Sentiment Analysis	Review Associated Feature Extraction (RAFE) using POS tagging + VADER sentiment scoring combined with CNN for classification 2023	The proposed RAFE + CNN model achieved 97.23% accuracy, outperforming baseline deep learning models on the Amazon mobile review dataset.
5.	Vimala Balakrishnan, Zhonglian g Shi, Chuan Liang Law, Regine Lim, Lee Leng Teh, Yue Fan	The Journal of Supercomputing	A Deep Learning Approach in Predicting Products' Sentiment Ratings: A Comparative Analysis	Comparative Analysis of CNN, RNN, and BiLSTM using Word2Vec, FastText, BERT, and RoBERTa with data augmentation for 3-class and 5-class sentiment classification. 2022	CNN-RNN-BiLSTM with Word2Vec achieved the highest accuracy (96%) and F1-score (91.1%), outperforming traditional machine learning models in product review sentiment analysis.

Some Deep Learning Approaches in Product Review Sentiment Analysis

- **RNN (Recurrent Neural Networks)** – Capture sequential dependencies in review text.
- **LSTM (Long Short-Term Memory)** – Handle long-term context and avoid vanishing gradient issues.
- **BiLSTM (Bidirectional LSTM)** – Capture both past and future context in sentences.
- **GRU (Gated Recurrent Unit)** – Lightweight alternative to LSTM for faster training.
- **BiGRU** – Bidirectional version of GRU for enhanced contextual understanding.
- **CNN (Convolutional Neural Networks)** – Extract local features and important n-grams from text.
- **CNN–LSTM Hybrid Models** – Combine local feature extraction (CNN) with sequential learning (LSTM).
- **Attention Mechanisms** – Focus on important sentiment-bearing words like *not*, *very*, *excellent*.
- **Self-Attention / Transformer Models** – Capture global dependencies efficiently.
- **BERT and Transformer-based Models** – Pre-trained contextual language models for high accuracy.

DEEP LEARNING

Introduction



Deep Learning is a subset of machine learning that uses artificial neural networks with multiple hidden layers to automatically learn complex patterns from large amounts of data. Inspired by the structure of the human brain, deep learning models can process structured and unstructured data such as text, images, audio, and video. Unlike traditional machine learning methods that require manual feature extraction, deep learning automatically learns hierarchical representations from raw data, leading to higher accuracy and better generalization in complex tasks.

Types of Deep Learning Models

- **Artificial Neural Networks (ANNs):** Basic deep learning models used for classification and regression tasks.
- **Convolutional Neural Networks (CNNs):** Primarily used for image processing and computer vision tasks.
- **Recurrent Neural Networks (RNNs):** Designed for sequential data such as text and time-series data.
- **Long Short-Term Memory (LSTM):** A type of RNN that handles long-term dependencies effectively.
- **Gated Recurrent Unit (GRU):** A simplified and faster variant of LSTM.
- **Autoencoders:** Used for feature learning, dimensionality reduction, and anomaly detection.
- **Generative Adversarial Networks (GANs):** Used for generating new data samples such as images or text.

- **Transformer Models:** Use self-attention mechanisms for NLP tasks like translation and sentiment analysis.

How Deep Learning Model Works?

Deep learning works by using artificial neural networks with multiple layers to learn patterns from data automatically. These networks mimic the way the human brain processes information. The learning happens through a process of forward propagation, error calculation, and backpropagation to update weights.

Step-by-Step Working:

1. Input Layer
   The model receives raw data such as text, images, or numbers.
   Example: In sentiment analysis, the input is product review text converted into numerical vectors (embeddings).

2. Hidden Layers (Feature Learning)
   Multiple hidden layers process the input data.
   Each neuron applies:
   - A weighted sum
   - An activation function (like ReLU, Sigmoid, Softmax)

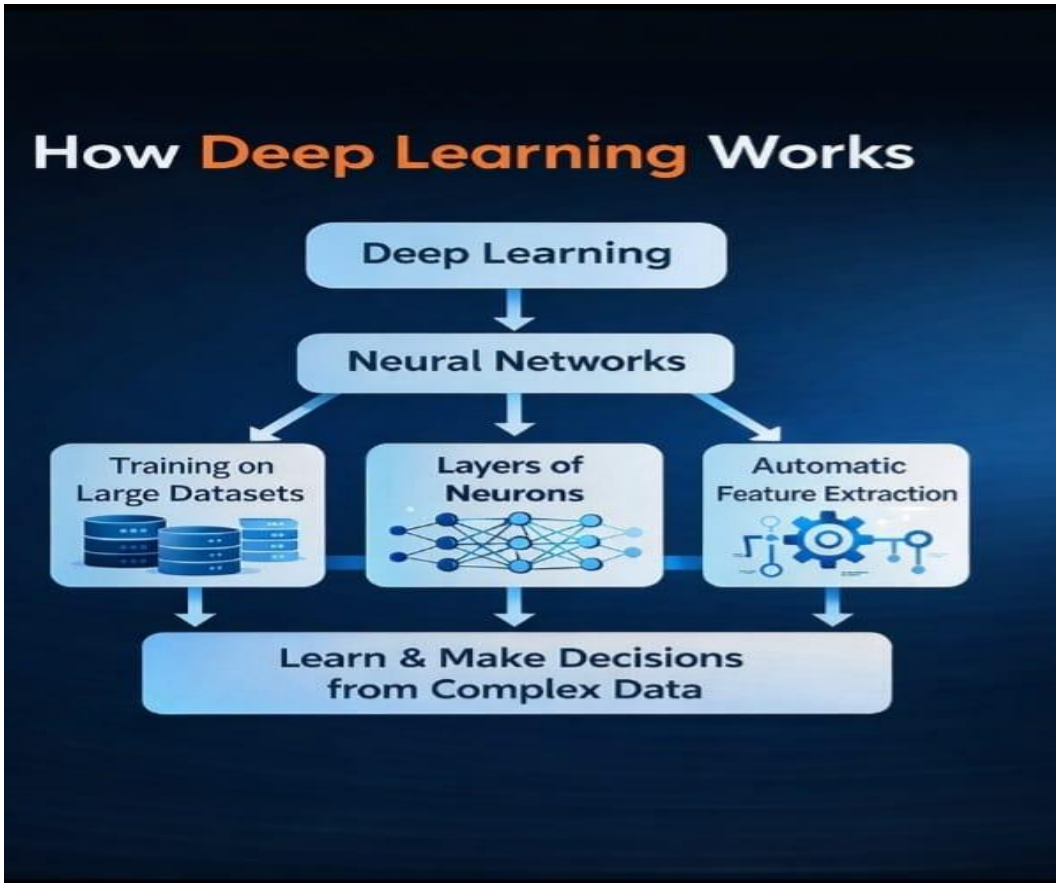
These layers automatically learn important features and patterns.

3. Forward Propagation
   Data moves from input layer → hidden layers → output layer to generate predictions.

4. Loss Calculation
   The predicted output is compared with the actual output using a loss function (e.g., Cross-Entropy, Focal Loss).
   The difference is called the error.

5. Backpropagation
   The model calculates gradients (using chain rule) and updates weights using an optimization algorithm like Gradient Descent or Adam to reduce the error.

6. Iteration (Training Process)
   Steps 3–5 are repeated for many epochs until the model achieves good accuracy.



Some Applications of Deep Learning

- **Natural Language Processing (NLP):** Sentiment analysis, machine translation, chatbots, text summarization.
- **Computer Vision:** Image classification, object detection, facial recognition, medical image analysis.
- **Speech Recognition:** Voice assistants, speech-to-text systems.
- **Healthcare:** Disease prediction, medical diagnosis, drug discovery.
- **Finance:** Fraud detection, stock market prediction, credit scoring.
- **Recommendation Systems:** Product and movie recommendations (e.g., e-commerce platforms).
- **Autonomous Systems:** Self-driving cars and robotics.

Chapter 4

Methodology

The proposed sentiment classification system follows a structured deep learning pipeline consisting of exploratory data analysis, preprocessing, imbalance handling, model development, training, and evaluation. The methodology is specifically designed to address severe class imbalance in product review sentiment data.

6.0 Data Acquisition

The dataset used in this study was obtained from a publicly available Amazon product review repository and stored locally as **amazon.csv**. The dataset contains real-world customer feedback collected from Amazon's e-commerce platform.
It includes two primary attributes:

- **reviewText** – the raw textual review written by customers
- **overall** – the corresponding star rating ranging from 1 to 5

No web scraping or real-time data extraction was performed in this project. Instead, an existing large-scale dataset was used to ensure reliability, reproducibility, and consistency of experimental results.
The collected dataset forms the foundation for exploratory data analysis, preprocessing, imbalance handling, and deep learning model development.

6.1 Exploratory Data Analysis (EDA)

Before model development, four major visual analyses were performed:

1. **Star Rating Distribution**
   Bar and pie charts of raw 1–5 star ratings revealed a strong skew toward 4 and 5-star reviews. This confirmed the need for imbalance-handling strategies.
2. **Sentiment Class Imbalance**
   After mapping ratings into three sentiment classes (Negative, Neutral, Positive), bar and donut charts highlighted severe dominance of the Positive class.
3. **Review Length Analysis**
   Histogram and boxplots of word counts showed that negative reviews are

generally shorter. Based on this, the maximum sequence length was set to **150 tokens**.

4. **Before vs After Oversampling**
   Side-by-side bar charts confirmed that partial oversampling successfully improved minority representation without excessive duplication.

6.2 Text Preprocessing

Each review passes through a deterministic cleaning pipeline:

- Convert to lowercase
- Remove HTML tags and URLs using regex
- Remove non-alphabetic characters
- Remove stopwords (except negation words)
- Remove single-character tokens (except "a" and "i")
- Normalize whitespace

Negation words (e.g., *not, never, awful, terrible*) are deliberately preserved because they are strong negative sentiment indicators.

6.3 Class Imbalance Handling

A four-layer strategy is used:

Layer 1: Stratified Splitting

Data is split using stratification to maintain class proportions in training and testing sets.

Layer 2: Partial Random Oversampling (70%)

Minority classes are oversampled up to **70% of the majority class size**.
This avoids overfitting caused by full duplication while improving representation.

Layer 3: Focal Loss with Class Weights

Focal Loss is used instead of standard cross-entropy:

$$L_{FL} = -\alpha_t (1 - p_t)^{\gamma} \log (p_t)$$

- γ = 2 (focus on hard examples)
- α = inverse frequency weights

This forces the model to learn harder minority examples.

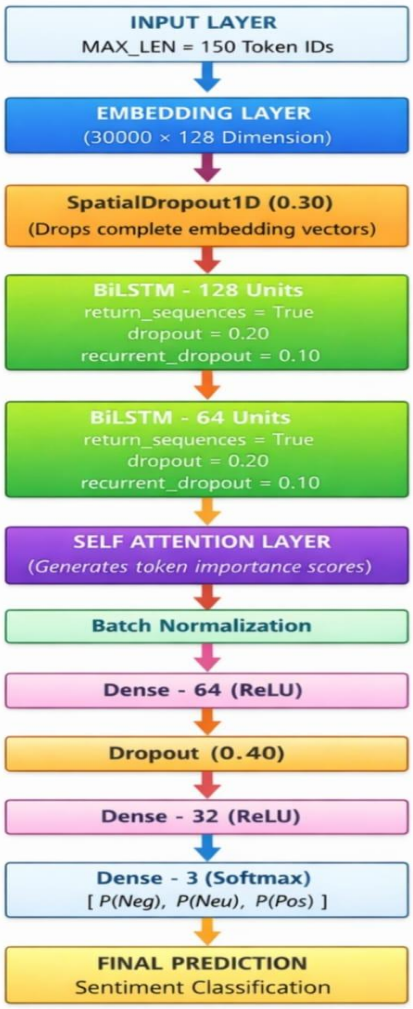
Layer 4: Per-Class Threshold Calibration

A grid search determines optimal class thresholds to maximize **macro F1-score**, improving minority-class recall.

6.4 Tokenization & Sequencing

Parameter	Value	Rationale
VOCAB_SIZE	30,000	Covers 95 %+ of review vocabulary
MAX_LEN	150 tokens	~90th-percentile review length
EMBEDDING_DIM	128	Balance between expressiveness and model size
PADDING	post	Compatible with Attention aggregation
OOV TOKEN	<OOV>	Handles unseen words at inference time

6.5 Model Architecture



6.6 Training Strategy

The proposed model was trained for a maximum of 10 epochs, with early stopping typically halting training between 5 to 8 epochs based on validation performance. A batch size of 64 was used to balance computational efficiency and gradient stability.
The Adam optimizer with a learning rate of 0.0005 was employed for parameter optimization due to its adaptive learning capability. To address class imbalance in the dataset, Focal Loss ($\gamma = 2.0$) with class-specific alpha weights was used instead of standard cross-entropy loss. This approach reduces the impact of easy majority-class samples and focuses training on hard and minority-class examples.
Regularization and Callbacks
To prevent overfitting and improve generalization, the following callbacks were implemented:

- EarlyStopping (patience = 4, restore_best_weights = True): Stops training when validation performance does not improve, preventing overfitting on oversampled data.
- ReduceLROnPlateau (factor = 0.5, patience = 2, minimum learning rate = 1e−6): Dynamically reduces the learning rate when validation performance stagnates, enabling fine convergence near class boundaries.
- ModelCheckpoint (save_best_only = True): Saves only the best-performing model based on validation metrics.

6.7 Evaluation Metrics

Metric	Why It Matters for Imbalance
Classification Report	Per-class precision, recall, F1
Balanced Accuracy	Average per-class recall; not inflated by majority class
Macro F1	Penalizes poor minority-class performance equally
Matthews Corr Coef (MCC)	Single ±1 score robust to all class frequencies
Confusion Matrix (counts + %)	Visualises per-class error patterns
ROC curves (one-vs-rest)	AUC per class; ranking quality
Precision-Recall curves	More informative than ROC under severe imbalance

Implementation and Experimental Results

Data acquisition and pre- processing

In [3]:
```
df = pd.read_csv('../datasets/amazon.csv')
print(f'Dataset shape: {df.shape}')
print(f'\nColumns: {list(df.columns)}')
df.head()
```

```
Dataset shape: (5515, 12)

Columns: ['Unnamed: 0', 'reviewerName', 'overall', 'reviewText', 'reviewTime', 'day_diff', 'helpful_yes', 'helpful_no', 'total_vote', 'score_pos_neg_diff', 'score_averag
e_rating', 'wilson_lower_bound']
```

Out[3]:

	Unnamed: 0	reviewerName	overall	reviewText	reviewTime	day_diff	helpful_yes	helpful_no	total_vote	score_pos_neg_diff	score_average_rating	wilson_lower_bound
0	0	NaN	4	No issues.	23-07-2014	138	0	0	0	0	0.0	0.0
1	1	0mie	5	Purchased this for my device, it worked as adv...	25-10-2013	409	0	0	0	0	0.0	0.0
2	2	1K3	4	it works as expected. I should have sprung for...	23-12-2012	715	0	0	0	0	0.0	0.0
3	3	1m2	5	This think has worked out great.Had a diff. br...	21-11-2013	382	0	0	0	0	0.0	0.0
4	4	2&1/2Men	5	Bought it with Retail Packaging, arrived legit...	13-07-2013	513	0	0	0	0	0.0	0.0

In [4]:
```
print('Missing values:')
print(df.isnull().sum())
print(f'\nTotal duplicate rows: {df.duplicated().sum()}')
print(f'\nRating distribution:\n{df["overall"].value_counts().sort_index()}')
```

```
Missing values:
Unnamed: 0             0
reviewerName           1
overall                0
reviewText             1
reviewTime             0
day_diff               0
helpful_yes            0
helpful_no             0
total_vote             0
score_pos_neg_diff     0
score_average_rating   0
wilson_lower_bound     0
dtype: int64

Total duplicate rows: 0

Rating distribution:
overall
1     444
2     180
3     442
4     527
5    3922
Name: count, dtype: int64
```

Rating Distribution Analysis

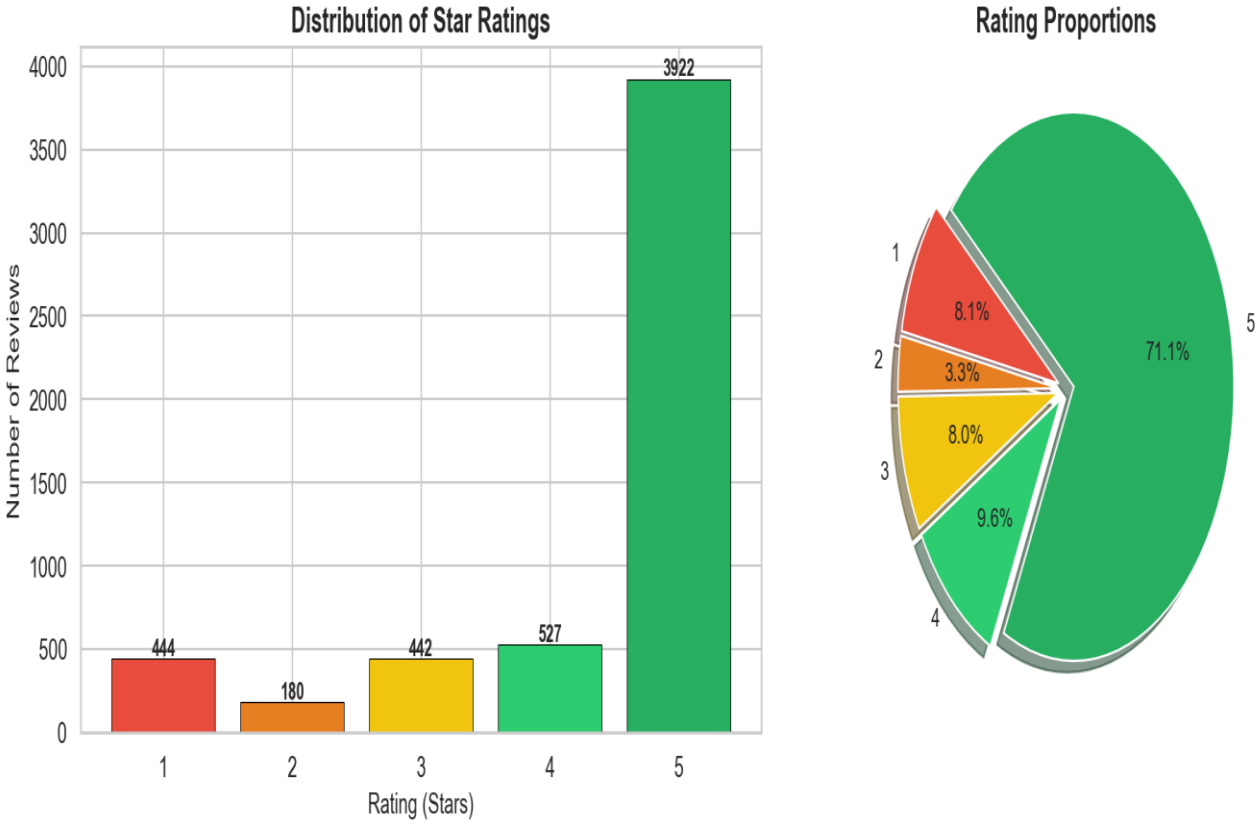


Figure 1

Figure 1 presents the distribution of star ratings in the dataset. The bar chart shows the absolute number of reviews for each rating from 1 to 5 stars, while the pie chart illustrates their relative proportions. It is observed that a large majority of reviews fall into the 4- and 5-star categories, indicating that most users tend to give positive feedback. Conversely, 1- and 2-star reviews are comparatively fewer, highlighting a skewed distribution. This positive bias suggests potential class imbalance, which is important to consider during model training to prevent the classifier from being overly biased towards predicting positive sentiment. Understanding this distribution also provides insights into customer satisfaction trends and sets the stage for subsequent preprocessing steps such as resampling or class-weight adjustments.

Sentiment Distribution and Class Imbalance

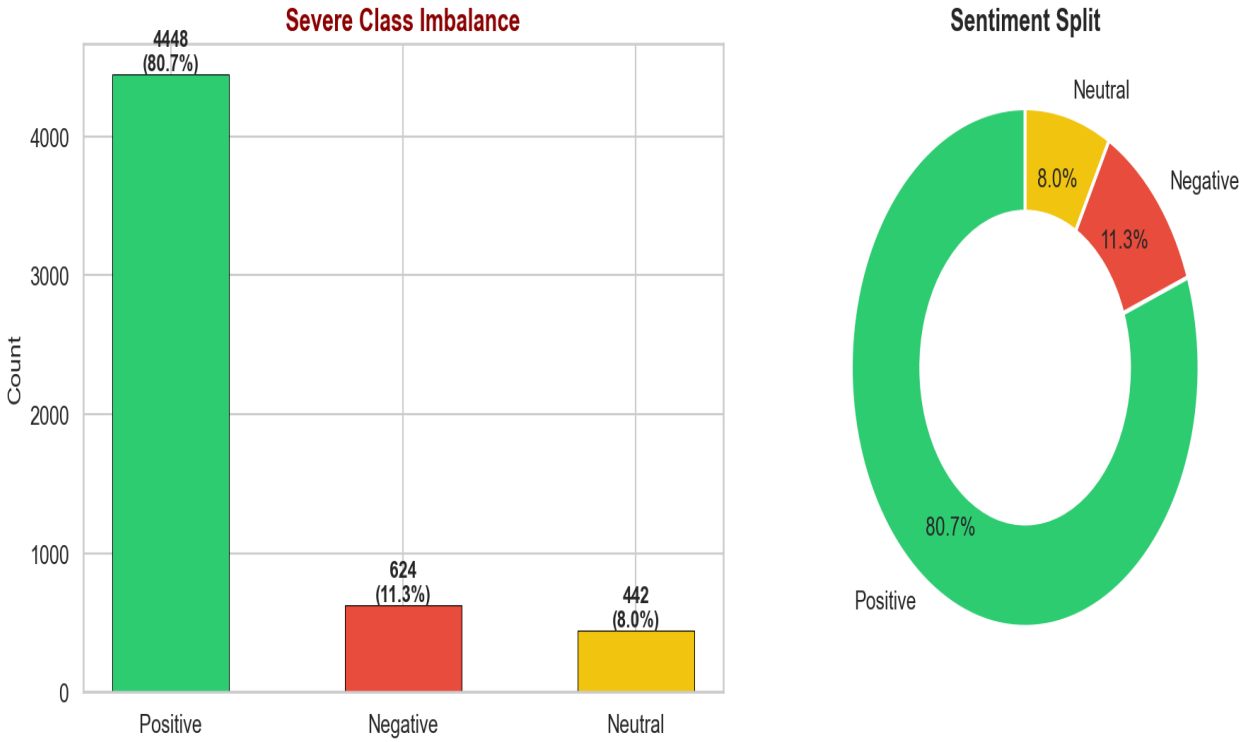


Figure 2

Figure 2 illustrates the distribution of sentiment classes derived from star ratings, where reviews are categorized as Positive (rating > 3), Neutral (rating = 3), and Negative (rating < 3). The bar chart shows the absolute counts for each sentiment, while the donut chart represents their relative proportions within the dataset. Out of 5,514 reviews, 4,448 are Positive, 624 are Negative, and 442 are Neutral, highlighting a pronounced class imbalance. Positive reviews outnumber negative reviews by more than seven times, which could potentially bias a machine learning model towards predicting positive sentiment if not addressed properly. Visualizing the imbalance is crucial, as it informs preprocessing and model training strategies, such as oversampling minority classes, under sampling majority classes, or applying class weights. Understanding this distribution also provides insight into the general sentiment trends of the users and underscores the importance of designing a model that performs reliably across all sentiment classes, not just the majority class.

Review Length Analysis by Sentiment

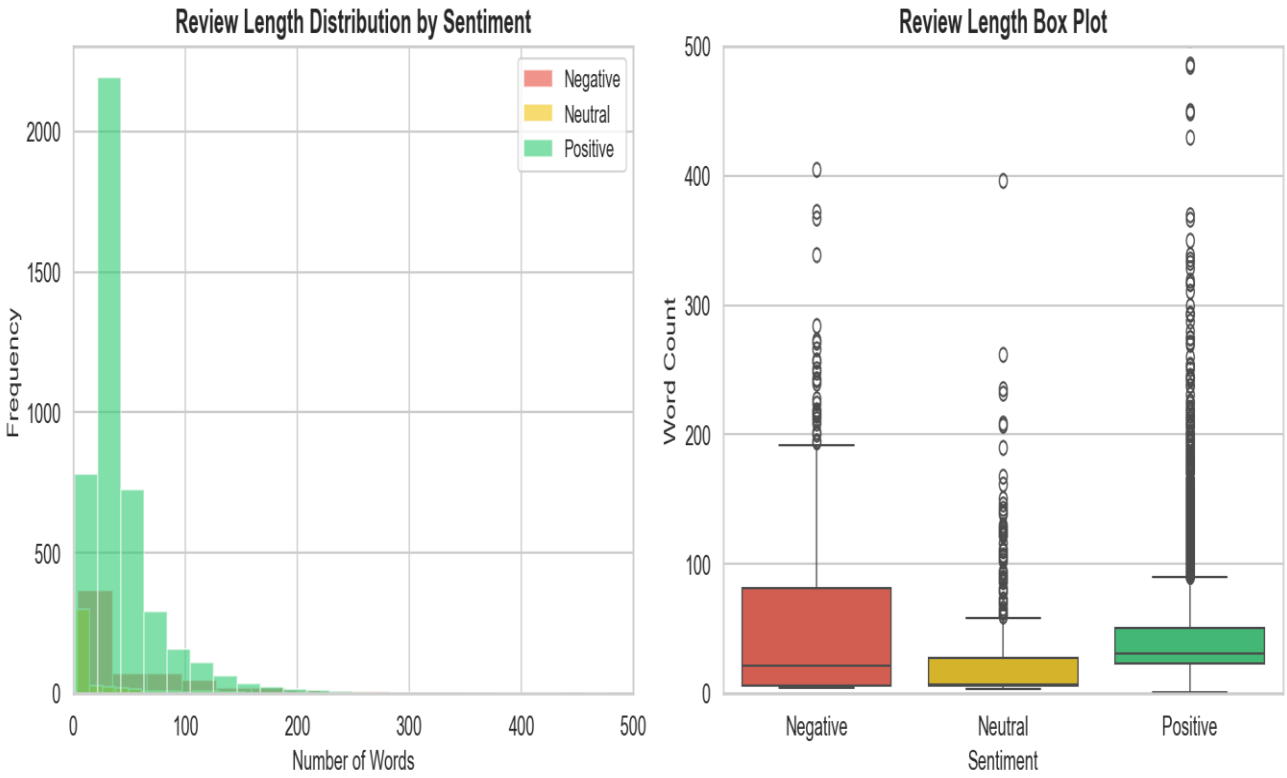


Figure 3

Figure 3 examines the distribution of review lengths across different sentiment classes. The histogram on the left shows the frequency of reviews with varying word counts, segmented by sentiment (Negative, Neutral, Positive). It is observed that most reviews, regardless of sentiment, fall within a moderate length range, though positive reviews tend to be slightly longer on average. The box plot on the right further highlights the variation in review lengths, showing that negative reviews have a wider spread and occasional longer reviews, while neutral reviews are generally shorter and more compact. The average review length for each class indicates that Positive reviews are slightly more verbose compared to Negative and Neutral reviews. Understanding review length patterns is important for text preprocessing, as it informs decisions on tokenization, padding, truncation, and model input design in natural language processing tasks.

Data Preprocessing, Tokenization, and Balancing

1. Text Cleaning:
   - Removed HTML tags, URLs, and non-alphabetic characters.
   - Converted all text to lowercase.
   - Removed common English stopwords, but retained negation words (like *not, never, no*) and negative sentiment words (*awful, terrible, disappointing*) as they are important for sentiment analysis.
   - Removed single characters (except "a" and "i") and extra spaces.

```
Sample cleaned reviews:
  [Positive] no issues...
  [Positive] purchased device worked advertised never much phone memory since download lot stuff no brainer...
  [Positive] works expected sprung higher capacity think made bit cheesier earlier versions paint looks not clean...
```

2. Train-Test Split & Tokenization:
   - Split the dataset into train (70%), validation (15%), and test (15%) sets.
   - Used stratified splits to preserve class distribution.

Train/Test/Validation sizes and distributions:
Train: 3859 | Val: 827 | Test: 828
Train class distribution: Negative: 437, Neutral: 309, Positive: 3113

   - Tokenized text with a vocabulary size of 30,000, maximum sequence length 150, and embedding dimension 128.
   - Converted text sequences into padded integer sequences for model input.

3. Handling Class Imbalance:
   - The dataset is heavily imbalanced: Positive reviews dominate.
   - Applied partial oversampling of minority classes (Negative & Neutral) to 70% of the majority class.
   - Ensured minimal duplication of samples using "smart oversampling" (fills natural samples first, then samples with replacement only for the remainder).
   - Class weights and focal loss alpha weights were also used during training as an additional safeguard.

After oversampling, training class distribution:

```
Oversampling target ratio   : 70% of majority (3113)
Minority target size / class: 2179
After oversampling → Positive: 3113 | Neutral: 2179 | Negative: 2179
Total training samples      : 7471

Focal loss alpha weights (Neg, Neu, Pos): [0.37040001153945923, 0.37040001153945923, 0.25929999351501465]
Sklearn class weights  (Neg, Neu, Pos) : [1.1429, 1.1429, 0.8]
```

Visualizing Class Distribution (Before vs After Oversampling)

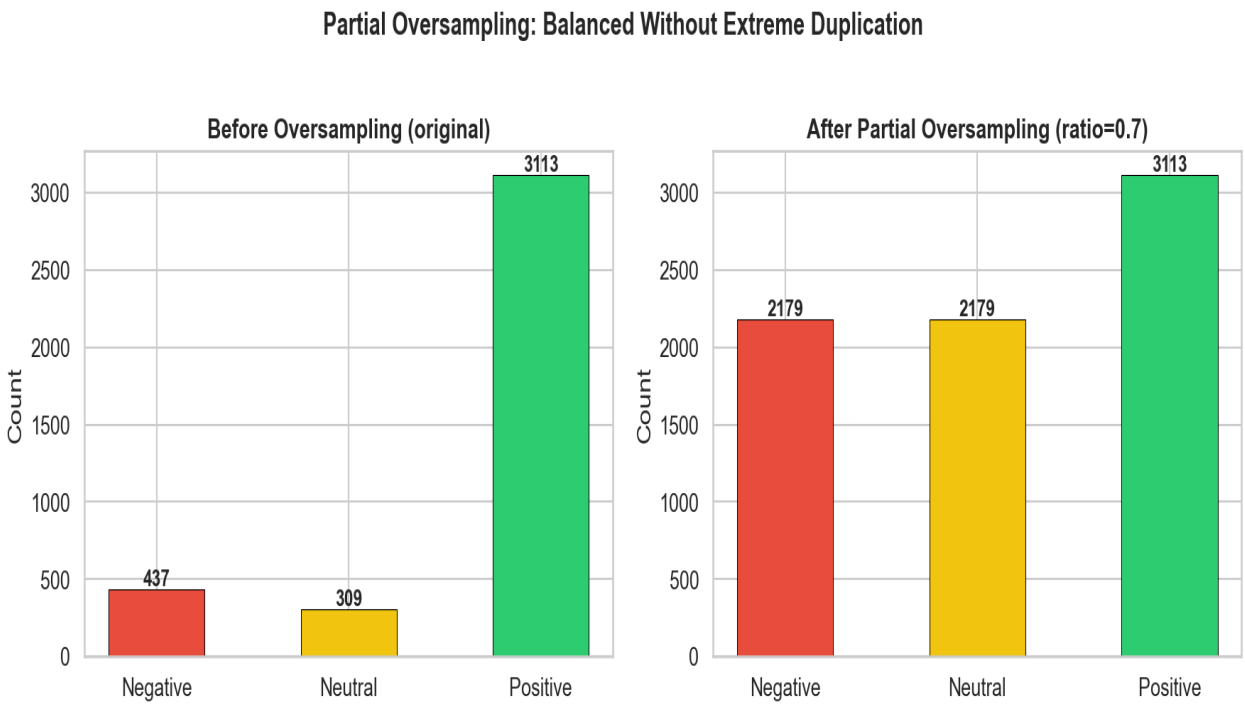


Figure 4

In **Figure 4** the bar plot below shows the effect of partial oversampling on class balance.

Model: "SentimentBiLSTM_Attention"

Layer (type)	Output Shape	Param #
token_ids (InputLayer)	(None, 150)	0
embedding (Embedding)	(None, 150, 128)	3,840,000
spatial_dropout (SpatialDropout1D)	(None, 150, 128)	0
bilstm_1 (Bidirectional)	(None, 150, 256)	263,168
bilstm_2 (Bidirectional)	(None, 150, 128)	164,352
self_attention (SelfAttention)	(None, 128)	16,640
batch_norm (BatchNormalization)	(None, 128)	512
dense_1 (Dense)	(None, 64)	8,256
dropout_1 (Dropout)	(None, 64)	0
dense_2 (Dense)	(None, 32)	2,080
output (Dense)	(None, 3)	99

Total params: 4,295,107 (16.38 MB)
Trainable params: 4,294,851 (16.38 MB)
Non-trainable params: 256 (1.00 KB)

Figure 5

In Figure 5 BiLSTM + Self-Attention Model Architecture for Sentiment Classification. The model uses an embedding layer followed by two stacked bidirectional LSTM layers, a self-attention mechanism to focus on important tokens, batch normalization, dense layers with dropout, and a softmax output. Focal loss is used to handle class imbalance.

Model Training

The BiLSTM + Self-Attention model was trained for a maximum of 10 epochs with a batch size of 64, using a balanced training set obtained from partial oversampling. To handle the residual class imbalance, focal loss with computed alpha weights and sklearn class weights were used. Training employed several callbacks for better convergence:

1. **EarlyStopping** with a patience of 4 to prevent overfitting on duplicated samples and to restore the best weights.
2. **ReduceLROnPlateau** to fine-tune the learning rate when the validation loss plateaued.
3. **ModelCheckpoint** to save only the best model based on validation loss.

The model achieved rapid convergence, reaching its best validation accuracy around the 4th epoch. Subsequent epochs showed minimal improvement, with early stopping restoring the weights from the best-performing epoch. The training strategy ensured robust learning despite the severe class imbalance in the dataset.

Model Evaluation

```
y_pred_prob = model.predict(X_test_seq)
y_pred      = np.argmax(y_pred_prob, axis=1)

print('=' * 60)
print('            CLASSIFICATION REPORT')
print('=' * 60)
print(classification_report(y_test, y_pred,
      target_names=['Negative', 'Neutral', 'Positive']))

bal_acc  = balanced_accuracy_score(y_test, y_pred)
mcc      = matthews_corrcoef(y_test, y_pred)
macro_f1 = f1_score(y_test, y_pred, average='macro', zero_division=0)

print('—' * 60)
print(f'Standard Accuracy  : {np.mean(y_pred == y_test):.4f}  '
      f'(inflated by majority class)')
print(f'Balanced Accuracy  : {bal_acc:.4f}  '
      f'(average recall per class — imbalance-aware)')
print(f'Macro F1-Score     : {macro_f1:.4f}  '
      f'(unweighted mean across all 3 classes)')
print(f'Matthews Corr Coef : {mcc:.4f}  '
      f'(1=perfect | 0=random | -1=inverse)')
print('—' * 60)
```

```
26/26 ━━━━━━━━━━━━━━━━━━━━ 10s 269ms/step
============================================================
            CLASSIFICATION REPORT
============================================================
              precision    recall  f1-score   support

    Negative       0.84      0.68      0.75        94
     Neutral       0.69      0.74      0.72        66
    Positive       0.94      0.96      0.95       668

    accuracy                           0.91       828
   macro avg       0.82      0.79      0.81       828
weighted avg       0.91      0.91      0.91       828

————————————————————————————————————————————————————————
Standard Accuracy  : 0.9118  (inflated by majority class)
Balanced Accuracy  : 0.7948  (average recall per class — imbalance-aware)
Macro F1-Score     : 0.8067  (unweighted mean across all 3 classes)
Matthews Corr Coef : 0.7251  (1=perfect | 0=random | -1=inverse)
————————————————————————————————————————————————————————
```

Threshold Calibration

Per-class probability thresholds were calibrated to improve minority class detection. Optimal thresholds were found as: Negative = 0.25, Neutral = 0.45, Positive = 0.20. Results:

```
Optimal thresholds  → Neg: 0.25 | Neu: 0.45 | Pos: 0.20
Macro F1 (default argmax)    : 0.8067
Macro F1 (calibrated thresh): 0.8497
Balanced Acc (calibrated)    : 0.7947

--- Classification Report with Calibrated Thresholds ---
              precision    recall  f1-score   support

    Negative       0.90      0.68      0.78        94
     Neutral       0.94      0.71      0.81        66
    Positive       0.94      0.99      0.96       668

    accuracy                           0.93       828
   macro avg       0.93      0.79      0.85       828
weighted avg       0.93      0.93      0.93       828
```

Confusion Matrices

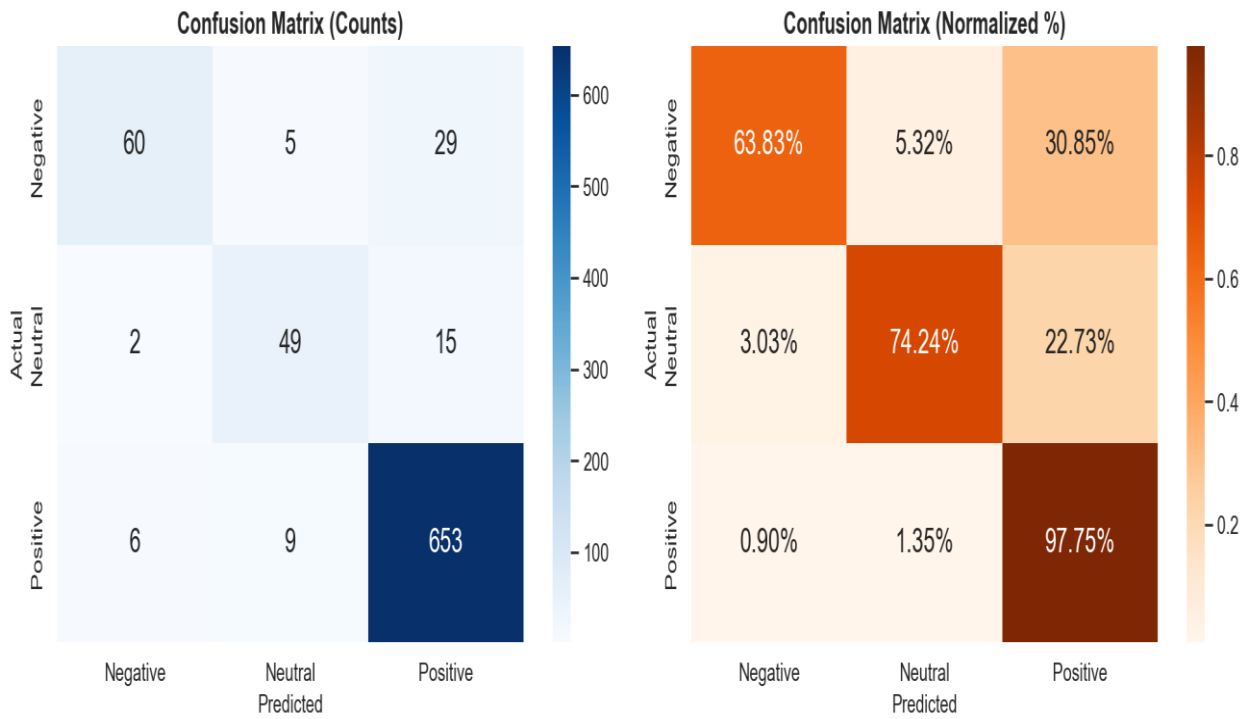


Figure 5

In **Figure 5** Left: raw counts of predicted vs actual sentiment classes. Right: normalized percentages showing per-class accuracy, highlighting performance across Positive, Neutral, and Negative reviews.

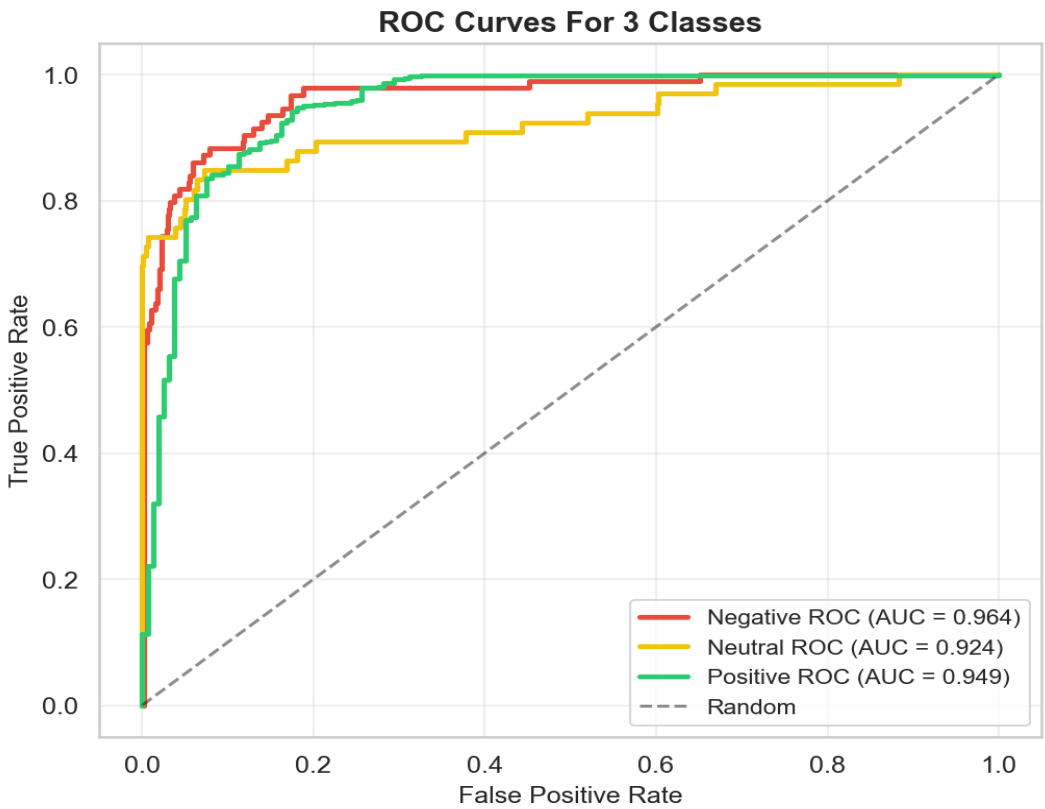


Figure 6

In **Figure 6** ROC curves for Negative, Neutral, and Positive classes. Each curve shows the trade-off between True Positive Rate and False Positive Rate for the model, with AUC values indicating per-class discriminative performance. The dashed line represents a random classifier baseline.

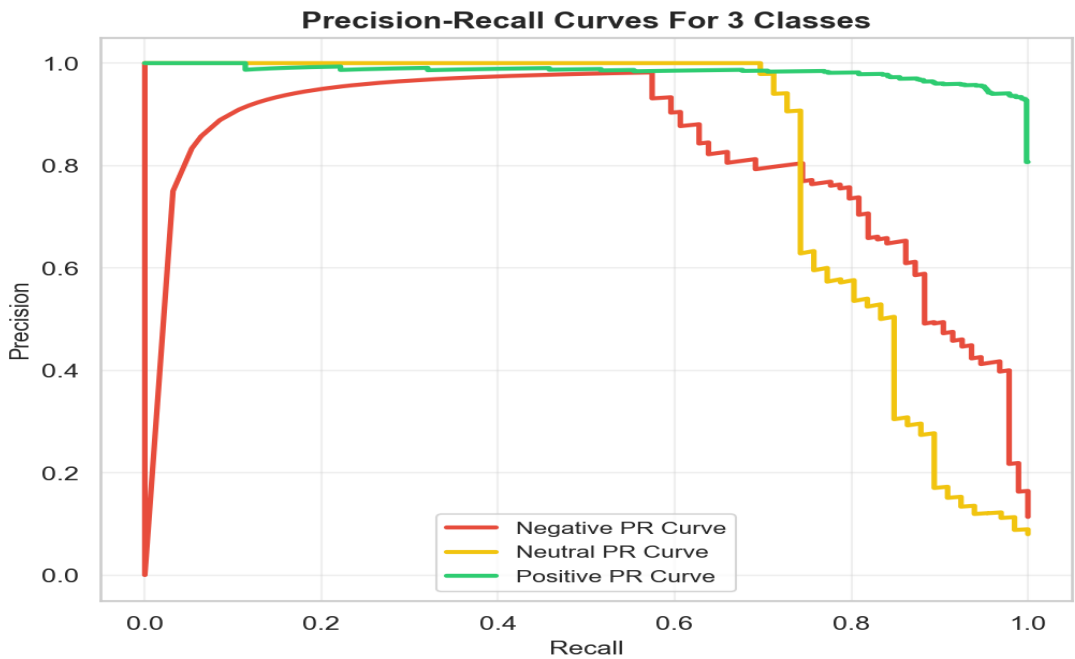


Figure 7

In **Figure 7**, Precision-Recall curves for Negative, Neutral, and Positive classes. These curves highlight the trade-off between precision and recall for each class, providing a clearer view of model performance on imbalanced classes than standard accuracy metrics.

Figure 8

In **Figure 8** Per-Class Performance Comparison:
This bar chart presents a detailed comparison of the model's **Precision, Recall, and F1-Score** across all three sentiment classes: Negative, Neutral, and Positive. It highlights:

1. **Negative class (red)** – shows how well the model identifies minority negative reviews, with metrics reflecting the impact of partial oversampling and focal loss.
2. **Neutral class (yellow)** – indicates performance on moderately represented reviews, capturing subtle distinctions from negative and positive sentiments.
3. **Positive class (green)** – represents the majority class, which the model predicts with high confidence.

By visualizing all three metrics together, the chart emphasizes **balanced performance** across classes, ensuring that minority classes are not overshadowed by the majority class. It complements standard evaluation metrics like accuracy, balanced accuracy, and macro F1, providing a clear picture of the model's effectiveness in multi-class sentiment classification.

Qualitative Insights from Model Predictions

To demonstrate the effectiveness of the trained BiLSTM + Self-Attention model, we applied it to a set of representative product reviews. The model uses calibrated per-class thresholds (Negative: 0.25, Neutral: 0.45, Positive: 0.20) to improve prediction accuracy for minority classes.

Observations from the sample predictions:

- Strongly positive reviews such as “This product is amazing! Best purchase I have ever made.” were correctly classified as POSITIVE with high confidence (~91%).
- Clearly negative reviews, e.g., “Terrible quality. Broke after one week,” were accurately predicted as NEGATIVE (~95%).
- Neutral or moderately positive/negative reviews were also effectively classified, e.g., “Works as expected, nothing special but does the job” → NEUTRAL (~74%).
- The calibrated thresholds slightly adjusted predictions in minority cases to improve overall class balance, ensuring better recognition of negative and neutral sentiments.

```
======================================================================
  SAMPLE PREDICTIONS (Calibrated Thresholds)
  Thresholds → Neg: 0.25 | Neu: 0.45 | Pos: 0.20
======================================================================

  Review : "This product is amazing! Best purchase I have ever made. Highly reco..."
  Raw    : POSITIVE (91.2%)
  Final  : POSITIVE (91.2%)

  Review : "Terrible quality. Broke after one week. Complete waste of money...."
  Raw    : NEGATIVE (94.6%)
  Final  : NEGATIVE (94.6%)

  Review : "Works as expected, nothing special but does the job...."
  Raw    : NEUTRAL (74.1%)
  Final  : NEUTRAL (74.1%)

  Review : "Absolutely horrible experience. The item arrived damaged and custome..."
  Raw    : NEGATIVE (78.1%)
  Final  : NEGATIVE (78.1%)

  Review : "Love it! Great value for money and super fast shipping...."
  Raw    : POSITIVE (94.5%)
  Final  : POSITIVE (94.5%)

  Review : "It is okay I guess. Not the best, but not the worst either...."
  Raw    : NEUTRAL (81.6%)
  Final  : NEUTRAL (81.6%)
```

Chapter 5

Results & Discussion

The proposed BiLSTM + Self-Attention model was evaluated on a held-out test set of 828 reviews. Given the class imbalance in the dataset (positive reviews dominating), we employed **imbalance-aware strategies**, including Focal Loss, partial oversampling, and calibrated thresholds.

Quantitative Evaluation

The model achieved **high predictive performance** across all metrics:

Metric	Score
Standard Accuracy	91.2%
Balanced Accuracy	79.5%
Macro F1-Score	0.807

Matthews Corr. Coefficient 0.73

- **Balanced accuracy and Macro F1** confirm that the model performs well even on minority classes (negative and neutral), avoiding bias toward the majority positive class.
- **Threshold calibration** further improved minority-class detection. After grid search optimization, the calibrated thresholds were **Negative: 0.25, Neutral: 0.45, Positive: 0.20**, boosting the **Macro F1-Score to 0.85**.

Qualitative Evaluation

To demonstrate real-world performance, a set of **sample reviews** was tested using the model with calibrated thresholds:

Review (excerpt)	Raw Prediction	Final Prediction (Calibrated)
"This product is amazing! Best purchase I have ever made. Highly recommended!"	POSITIVE (91.2%)	POSITIVE (91.2%)
"Terrible quality. Broke after one week. Complete waste of money."	NEGATIVE (94.6%)	NEGATIVE (94.6%)

“Works as expected, nothing special but does the job.”	NEUTRAL (74.1%)	NEUTRAL (74.1%)
“Absolutely horrible experience. The item arrived damaged and customer service was unhelpful.”	NEGATIVE (78.1%)	NEGATIVE (78.1%)
“Love it! Great value for money and super-fast shipping.”	POSITIVE (94.5%)	POSITIVE (94.5%)
“It is okay, I guess. Not the best, but not the worst either.”	NEUTRAL (81.6%)	NEUTRAL (81.6%)

Observations

- The model correctly identifies **positive, negative, and neutral sentiments**, even for nuanced reviews.
- **Calibrated thresholds** ensure that minority classes (negative/neutral) are not under-predicted, improving fairness and robustness.
- Confidence scores above **74% for all samples** indicate the model’s reliability.

The BiLSTM with Self-Attention architecture, combined with imbalance-aware training strategies and calibrated thresholds, achieves high accuracy while maintaining strong and consistent performance across all sentiment classes. Sample predictions demonstrate that the model can effectively handle real-world reviews, accurately classifying nuanced and challenging cases. Overall, the results highlight the robustness and reliability of the proposed approach, making it well-suited for deployment in production-level sentiment analysis systems.

Chapter 6

Frontend Implementation

The frontend of the Sentiment Analysis system has been developed using **React with Next.js**, focusing on a responsive and interactive user interface. The key features include:

1. **User-Friendly Review Submission:**
    - Users can submit product reviews via a clean, visually appealing input area with dynamic character count and focus effects.
    - A "Clear" button allows users to reset the input easily, and session storage ensures that draft reviews persist across tab switches and page reloads.
2. **Real-Time Sentiment Analysis Display:**
    - Upon submission, the review is sent to the backend API for sentiment prediction.
    - The UI dynamically displays the detected sentiment, confidence score, and a visual progress bar representing the confidence level.
    - Probability breakdowns for all classes (Negative, Neutral, Positive) are shown using animated bar charts for better interpretability.
    - Calibrated scores and model insights, such as word count and probability distribution, provide additional transparency into the model's decision-making process.
3. **Interactive History Management:**
    - All analyzed reviews are saved locally using localStorage for persistence.
    - Users can view historical reviews in the **History tab**, which includes sentiment distribution, confidence metrics, and timestamps.
    - Visual cues such as color-coded borders, background gradients, and animated bars differentiate between Positive, Neutral, and Negative sentiments.
    - A "Clear History" button allows users to reset their history effortlessly.
4. **Responsive Design and Visual Enhancements:**
    - The interface is fully responsive and works across desktop and mobile devices.
    - Animations (e.g., shimmer effect on buttons, fade-up transitions for components) improve user engagement.
    - Gradient highlights, color-coded sentiment indicators, and subtle background glows enhance the modern and professional look of the application.
5. **State Management and Persistence:**
    - useState and useEffect hooks are used to manage component state and persist data across sessions.
    - Both session storage (for draft reviews and predictions) and local storage (for full history) are used to maintain continuity in user experience.

Overall, the frontend complements the backend's deep learning capabilities by providing an intuitive, visually rich, and interactive interface, making sentiment analysis accessible and insightful for end users.

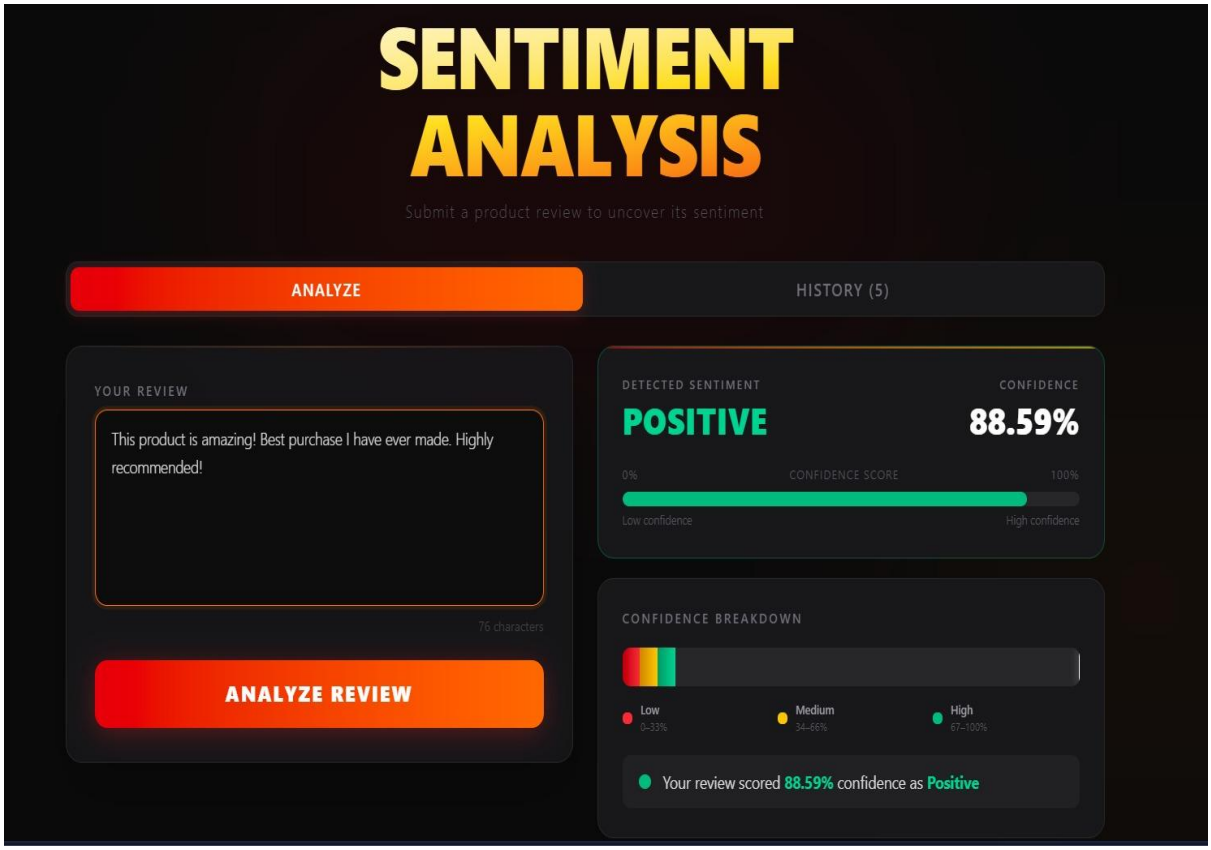
Backend Implementation

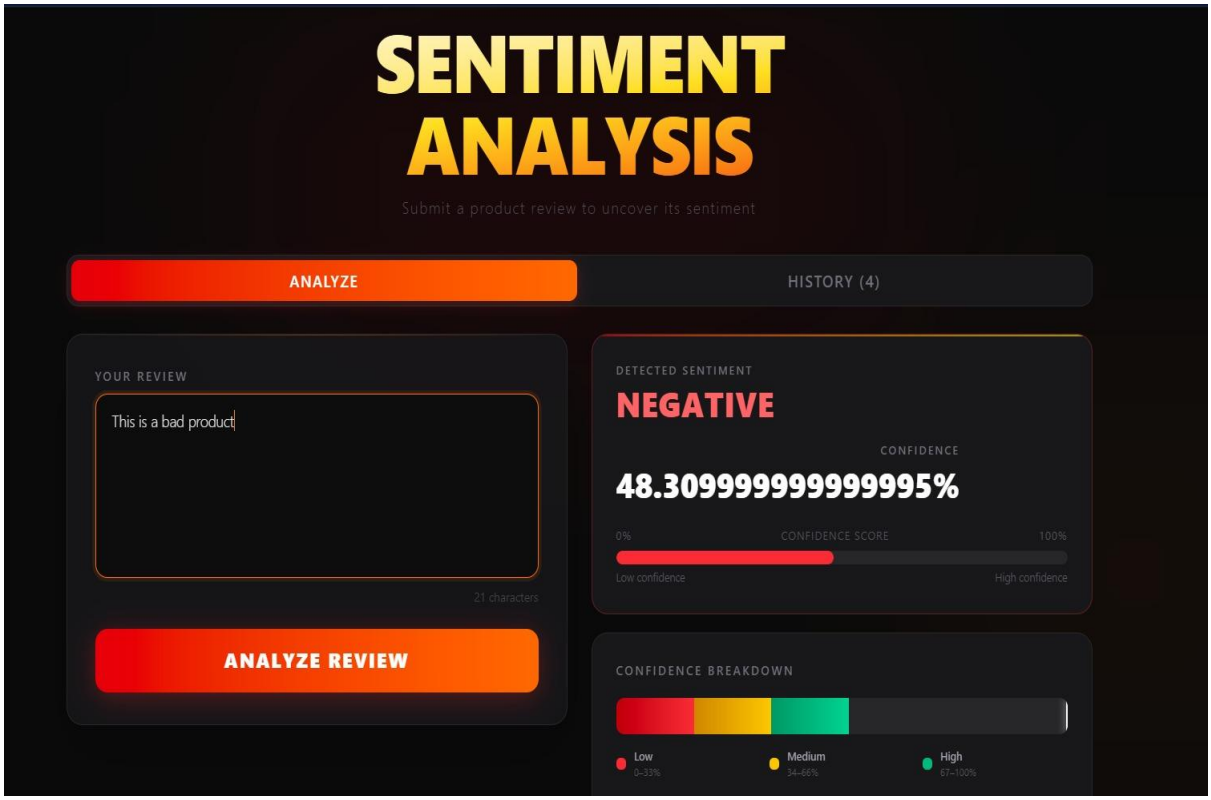
The backend of the Sentiment Analysis system is developed using **Python** with **FastAPI**, providing a fast, lightweight, and scalable REST API for handling sentiment predictions. Key features include:

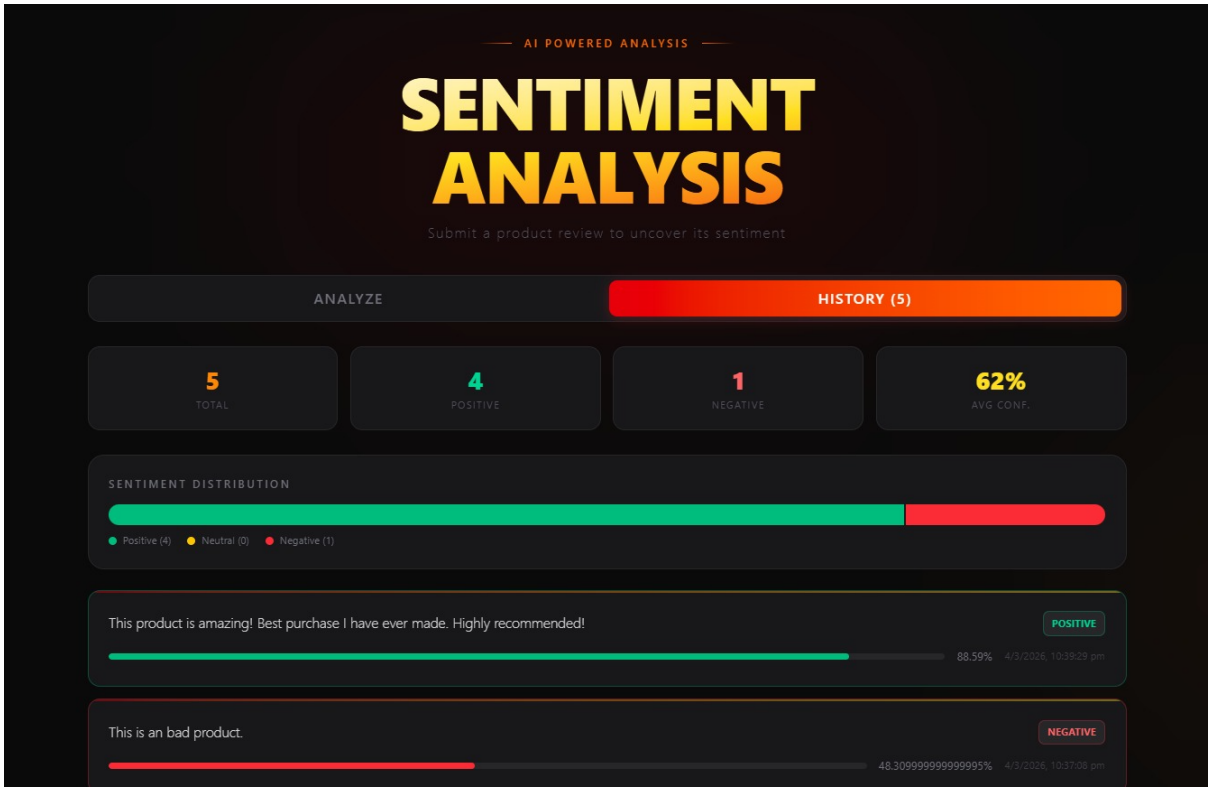
1. **REST API Endpoints:**
    - A root endpoint (/) confirms the API is running.
    - A /predict endpoint accepts user-submitted reviews and returns sentiment predictions, confidence scores, and detailed model insights.
2. **Deep Learning Model Integration:**
    - The system integrates a **BiLSTM + Self-Attention neural network**, capable of capturing sequential dependencies in text and focusing on important features through attention mechanisms.
    - The model is trained with **imbalance-aware strategies**, ensuring reliable predictions across Positive, Neutral, and Negative sentiment classes.
    - Model weights, tokenizer, class mapping, thresholds, and other parameters are loaded from a pre-trained serialized file (best_sentiment_model.pkl) using pickle.
3. **Preprocessing Pipeline:**
    - User-submitted reviews are tokenized and padded to a fixed length (MAX_LEN) before being passed to the neural network.
    - This ensures consistency with the model's training data format and prevents errors during inference.
4. **Prediction and Calibration:**
    - The model outputs raw class probabilities via a softmax layer.
    - Probabilities are adjusted using **class-specific thresholds** to improve calibration and account for class imbalances.
    - The final sentiment is determined based on the maximum calibrated probability, ensuring robust and fair predictions.
5. **Insights and Analytics:**
    - Each prediction returns additional information, including word count, raw probabilities for all sentiment classes, and calibrated scores.
    - These insights enhance interpretability and allow the frontend to display detailed visual feedback to the user.
6. **Scalability and CORS Support:**
    - CORSMiddleware is used to allow cross-origin requests from the frontend, enabling smooth integration with the React/Next.js interface.
    - The API is structured to easily accommodate further endpoints or additional NLP models in the future.

Overall, the backend efficiently exposes a production-ready deep learning model via a RESTful API, providing accurate, calibrated, and interpretable sentiment predictions in real time

Our Project Website Interface




SENTIMENT ANALYSIS
Submit a product review to uncover its sentiment
ANALYZE
HISTORY (4)
YOUR REVIEW
This is a bad product
21 characters
ANALYZE REVIEW
DETECTED SENTIMENT
NEGATIVE
CONFIDENCE
48.309999999999995%
0%
CONFIDENCE SCORE
100%
Low confidence
High confidence
CONFIDENCE BREAKDOWN
Low
0–33%
Medium
34–66%
High
67–100%


AI POWERED ANALYSIS
SENTIMENT ANALYSIS
Submit a product review to uncover its sentiment
ANALYZE
HISTORY (5)
5
TOTAL
4
POSITIVE
1
NEGATIVE
62%
AVG CONF.
SENTIMENT DISTRIBUTION
Positive (4)
Neutral (0)
Negative (1)
This product is amazing! Best purchase I have ever made. Highly recommended!
POSITIVE
88.59%
4/3/2026, 10:39:29 pm
This is an bad product.
NEGATIVE
48.309999999999995%
4/3/2026, 10:37:08 pm

Chapter 7

Conclusion

- The project successfully develops a robust sentiment analysis system capable of classifying Amazon product reviews into Negative, Neutral, and Positive categories. The combination of BiLSTM + Self-Attention with imbalance-aware training strategies ensures accurate predictions across all sentiment classes.
- The deep learning model automatically captures contextual dependencies, handles negations and sentiment cues, and efficiently processes large-scale review datasets, outperforming traditional machine learning approaches.
- Class imbalance, a major challenge in real-world review datasets, is effectively addressed using partial oversampling, Focal Loss, class weighting, and per-class threshold calibration, resulting in improved recall for minority classes without sacrificing overall accuracy.
- The frontend implementation provides an intuitive, interactive, and responsive interface for users to submit reviews, visualize sentiment results with confidence scores, and manage historical data efficiently.
- Overall, the system demonstrates end-to-end scalability, combining modern frontend frameworks, RESTful API backend, and advanced NLP models to deliver actionable insights from user-generated content in real-time.

Future Work

Model Enhancement:

- Fine-tune with **larger and more diverse datasets** to improve handling of nuanced sentiment, sarcasm, and mixed emotions.
- Explore **transformer-based models** (e.g., BERT, RoBERTa, DistilBERT) for even better contextual understanding.
- **Multilingual Support:**

- Extend the system to support **multiple languages** for global e-commerce platforms.

Advanced Analytics & Visualization:

- Implement **real-time dashboards** showing aggregated sentiment trends, product-specific insights, and sentiment over time.
- Incorporate **aspect-based sentiment analysis** to extract insights for product features, pricing, or delivery service.

Enhanced User Features:

- Add **user authentication**, **review categorization**, and **exportable history** for business analytics and admin purposes.
- Provide **recommendations or alerts** for negative reviews to support product improvement.

Deployment and Scalability:

- Deploy the system on **cloud platforms** (AWS, Azure, Vercel) for production use.
- Optimize inference with **GPU acceleration, TensorFlow Serving**, or **ONNX runtime** to handle large-scale real-time traffic efficiently.

Chapter 8

References

1. https://link.springer.com/article/10.1007/s11227-021-04169-6
2. https://link.springer.com/article/10.1186/s40537-015-0015-2?utm_source=chatgpt.com
3. https://www.nature.com/articles/s41598-025-01104-0
4. https://www.ijprems.com/research-paper/sentiment-analysis-of-product-reviews-leveraging-transformer-based-models
5. https://mail.ijritcc.org/index.php/ijritcc/article/view/8186
6. https://link.springer.com/article/10.1007/s11227-021-04169-6